

**LAPORAN PRAKTIKUM
TEKNOLOGI IOT II**



DI SUSUN OLEH :

NAMA : DEA ZUL HAFNI SHERLYZA

NPM : 25782065

KELAS : TRI 3 C

**PRODI TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
2026**

Modul 5: Topologi Jaringan Terdesentralisasi (ESP-NOW dan Mesh)

5.3 Teori Dasar

5.3.1 Jaringan Berbasis Infrastruktur vs Jaringan Terdesentralisasi

Pada jaringan berbasis infrastruktur, setiap perangkat *Station* (STA) harus tersambung ke *Access Point* (AP) pusat. Jika dua node ingin bertukar data, paket harus melewati *router* terlebih dahulu.

Kelemahan arsitektur terpusat untuk perangkat IoT lapangan:

1. Titik Kegagalan Tunggal (*Single Point of Failure*): Jika daya router padam atau sinyalnya terganggu, seluruh komunikasi sensor terputus seketika.
2. Konsumsi Daya & Latensi Tinggi: Proses asosiasi Wi-Fi (pemberian SSID, autentikasi WPA2 4-way handshake, serta negosiasi alamat IP via DHCP) memakan waktu 2 hingga 10 detik, yang sangat mengurangi kapasitas baterai.
3. Batas Jangkauan Radio Statis: Perangkat hanya dapat berkomunikasi selama masih berada dalam lingkaran jangkauan radio pemancar *router*.

Sebagai solusinya, arsitektur Peer-to-Peer (P2P) dan Mesh membebaskan perangkat dari *router*, memun

5.3.3 Konsep Struktur Data (struct) dalam Transmisi Paket

Transmisi paket ESP-NOW mengirimkan deretan bit memori mentah (*raw bytes*) melalui pointer data (`uint8_t *`). Agar beragam tipe data (seperti bilangan bulat, desimal, teks, dan status sakelar) dapat dikirimkan secara ringkas dalam satu kesatuan, bahasa C++ menyediakan fitur struct (struktur):

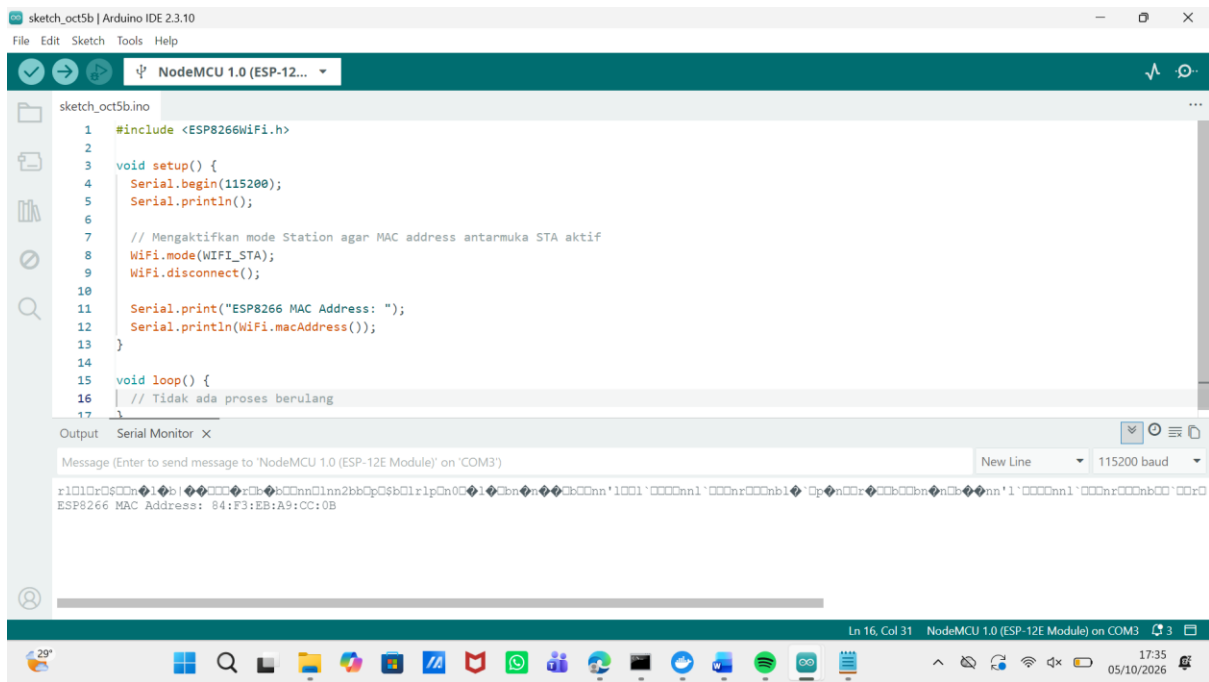
'_' allowed only in math mode

$$\text{Total Ukuran Payload} = \sum \text{sizeof}(\text{tipe_data}) \leq 250 \text{ byte}$$

5.5 Praktikum

5.5.1 Praktikum 1: Identifikasi MAC Address Perangkat NodeMCU

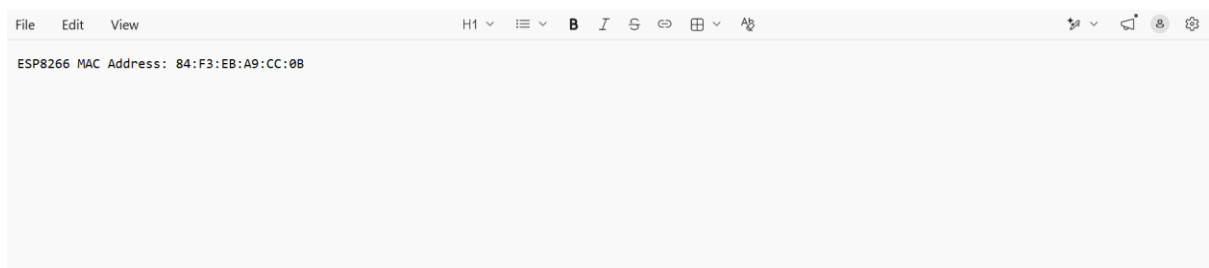
Sebelum membangun jaringan ESP-NOW, setiap anggota kelompok wajib membaca dan mencatat alamat fisik (*MAC Address*) dari masing-masing *board* NodeMCU.



Penjelasan Singkat Kode:

- `WiFi.mode(WIFI_STA);` : Menyiapkan modul radio Wi-Fi internal ke dalam mode *Station* agar alamat perangkat keras (*Hardware MAC*) siap dialokasikan oleh SDK.
- `WiFi.disconnect();` : Memastikan mikrokontroler tidak mencoba melakukan proses penyambungan ke jaringan luar.
- `WiFi.macAddress();` : Mengambil string alamat fisik 48-bit bawaan pabrik yang bersifat unik secara global.

Kemudian simpan MAC ADDRESS KE NOTEPAD :

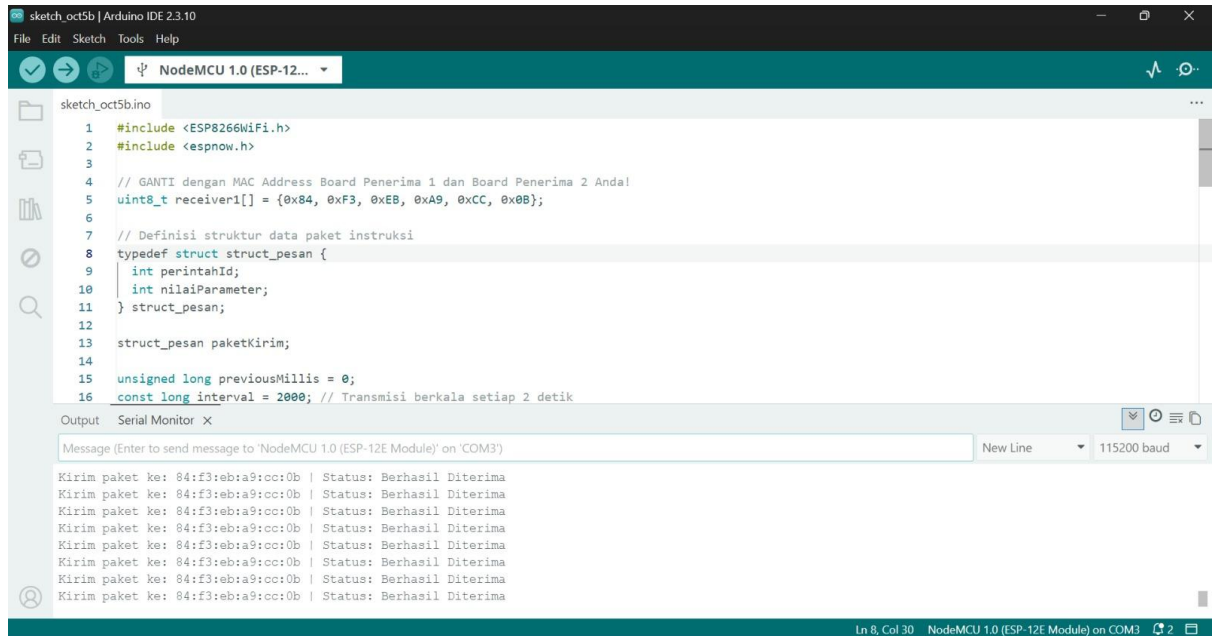


5.5.2 Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)

Skenario: Satu unit NodeMCU bertindak sebagai Controller / Sender, menyiarkan paket data instruksi yang sama secara langsung ke dua unit NodeMCU lain yang bertindak sebagai Slave / Receiver tanpa perantara router.

Langkah Kerja Pemrograman Sisi Pengirim (Controller):

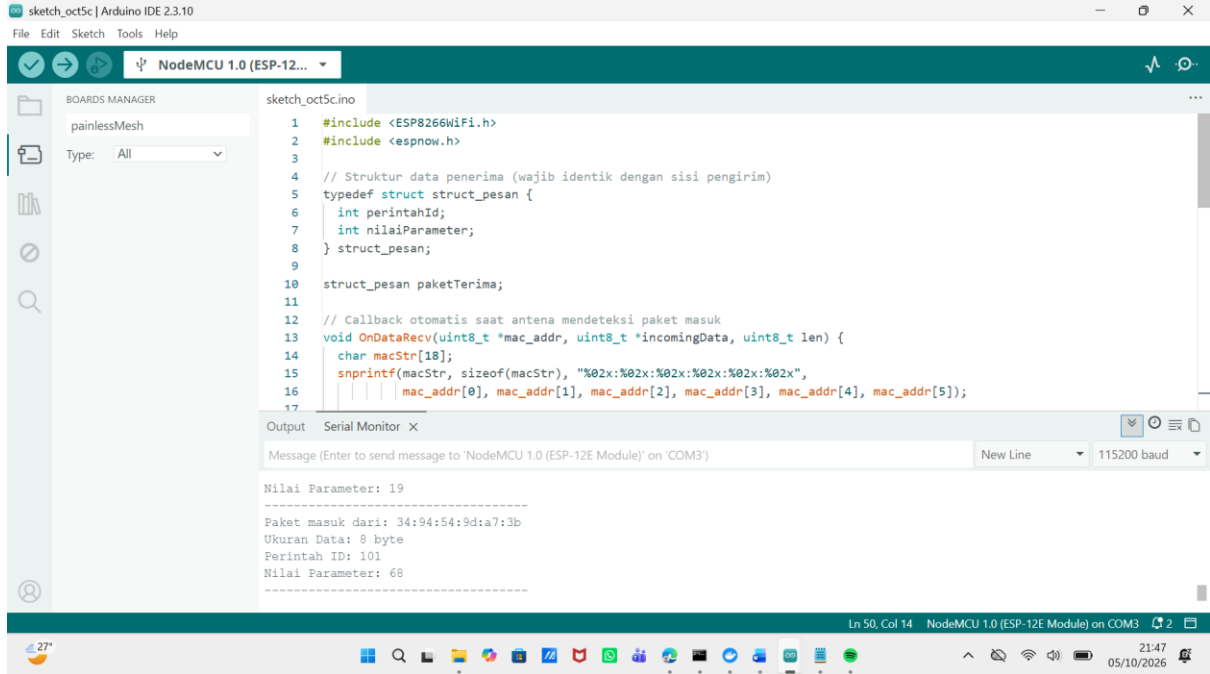
1. Masukkan alamat MAC milik Board Penerima 1 dan Board Penerima 2 ke dalam variabel receiver1 dan receiver2.
2. Format tanda titik dua (:) diubah menjadi bilangan heksadesimal berawalan 0x yang dipisahkan oleh tanda koma (contoh: BC:FF:4D:23:45:90 → {0xBC, 0xFF, 0x4D, 0x23, 0x45, 0x90}).



```
sketch_oct5b | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...)
sketch_oct5b.ino
1 #include <ESP8266Wifi.h>
2 #include <espnow.h>
3
4 // GANTI dengan MAC Address Board Penerima 1 dan Board Penerima 2 Andai
5 uint8_t receiver1[] = {0x84, 0xF3, 0xEB, 0xA9, 0xCC, 0x0B};
6
7 // Definisi struktur data paket instruksi
8 typedef struct struct_pesan {
9     int perintahId;
10    int nilaiParameter;
11 } struct_pesan;
12
13 struct_pesan paketKirim;
14
15 unsigned long previousMillis = 0;
16 const long interval = 2000; // Transmisi berkala setiap 2 detik
Output Serial Monitor X
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')
New Line 115200 baud
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Berhasil Diterima
Ln 8, Col 30 NodeMCU 1.0 (ESP-12E Module) on COM3 2
```

Langkah Kerja Pemrograman Sisi Penerima (Slave):

1. Unggah kode sketsa berikut ke NodeMCU 2 dan NodeMCU 3.
2. Board penerima tidak perlu mengetahui alamat MAC milik Controller.



```
sketch_oct5c.ino
1  #include <ESP8266WiFi.h>
2  #include <espnow.h>
3
4  // Struktur data penerima (wajib identik dengan sisi pengirim)
5  typedef struct struct_pesan {
6      int perintahId;
7      int nilaiParameter;
8  } struct_pesan;
9
10 struct_pesan paketTerima;
11
12 // Callback otomatis saat antenna mendeteksi paket masuk
13 void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
14     char macStr[18];
15     snprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x:%02x",
16             mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);
17
18     Serial.println("Paket masuk dari: " + String(macStr));
19     Serial.println("Ukuran Data: " + String(len) + " byte");
20     Serial.println("Perintah ID: " + String(paketTerima.perintahId));
21     Serial.println("Nilai Parameter: " + String(paketTerima.nilaiParameter));
22 }
```

Output Serial Monitor

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

New Line 115200 baud

Nilai Parameter: 19

Paket masuk dari: 34:94:54:9d:a7:3b

Ukuran Data: 8 byte

Perintah ID: 101

Nilai Parameter: 68

Hasil Praktikum :



Komunikasi One-to-Many ESP-NOW: Protokol ESP-NOW terbukti mampu menghubungkan satu perangkat pengirim (*Controller*) ke beberapa perangkat penerima (*Slave*) secara nirkabel tanpa memerlukan perantara router Wi-Fi.

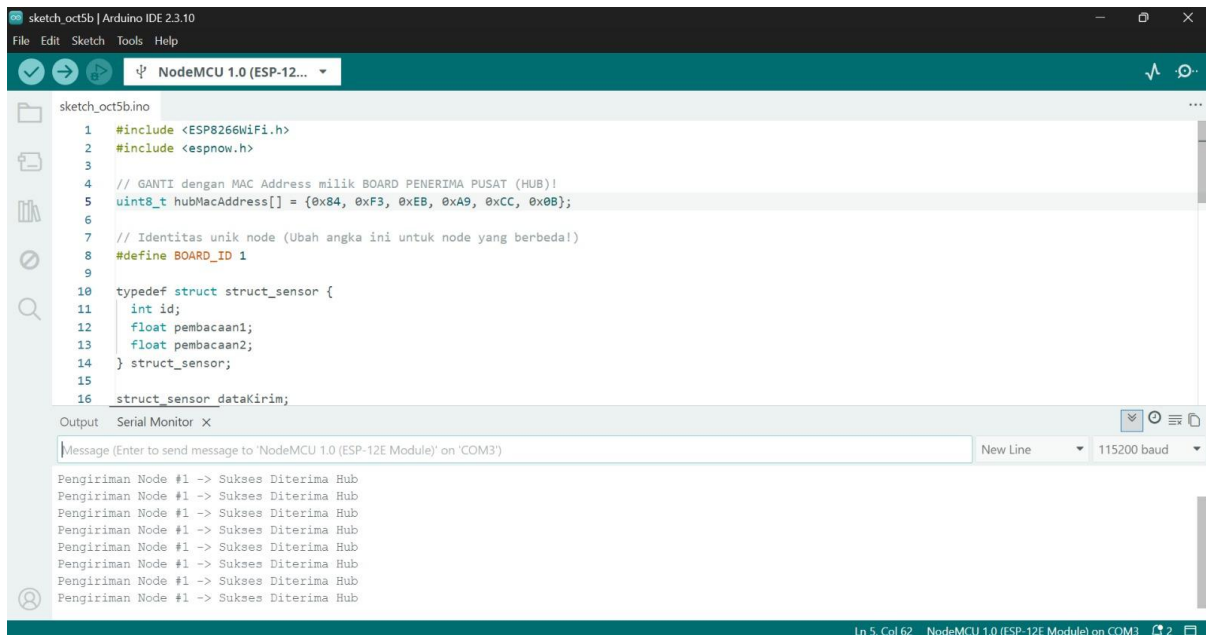
Mekanisme Pengiriman (*Controller*): Pengirim mendaftarkan alamat MAC masing-masing penerima ke dalam tabel memori internal menggunakan `esp_now_add_peer()`, kemudian menyiarkan data secara berurutan menggunakan argumen 0 pada fungsi `esp_now_send()`. Mekanisme Penerimaan (*Slave*): Perangkat penerima bekerja secara pasif dengan mengandalkan fungsi *callback* interupsi `OnDataRecv()` yang langsung aktif menangkap dan menyalin data mentah menggunakan `memcpy()` setiap kali gelombang radio mendeteksi paket masuk dari pengirim

5.5.3 Praktikum 3: Pengumpulan Data Telemetry Many-to-One (ESP-NOW)

Skenario: Dua node sensor di lapangan memancarkan data pengukuran masing-masing ke sebuah node konsentrator pusat (*Aggregator Hub / Base Station*).

Langkah Kerja Pemrograman Sisi Node Pengirim:

1. Masukkan alamat MAC milik Board Penerima Pusat (Hub) pada variabel `hubMacAddress`.
2. Pada Board Pengirim 1, atur: `#define BOARD_ID 1`.
3. Pada Board Pengirim 2, atur: `#define BOARD_ID 2`.



```
sketch_oct5b.ino
1  #include <ESP8266WiFi.h>
2  #include <espnow.h>
3
4  // GANTI dengan MAC Address milik BOARD PENERIMA PUSAT (HUB)!
5  uint8_t hubMacAddress[] = {0x84, 0xF3, 0xEB, 0xA9, 0xCC, 0x0B};
6
7  // Identitas unik node (Ubah angka ini untuk node yang berbeda!)
8  #define BOARD_ID 1
9
10 typedef struct struct_sensor {
11     int id;
12     float pembacaan1;
13     float pembacaan2;
14 } struct_sensor;
15
16 struct_sensor dataKirim;
```

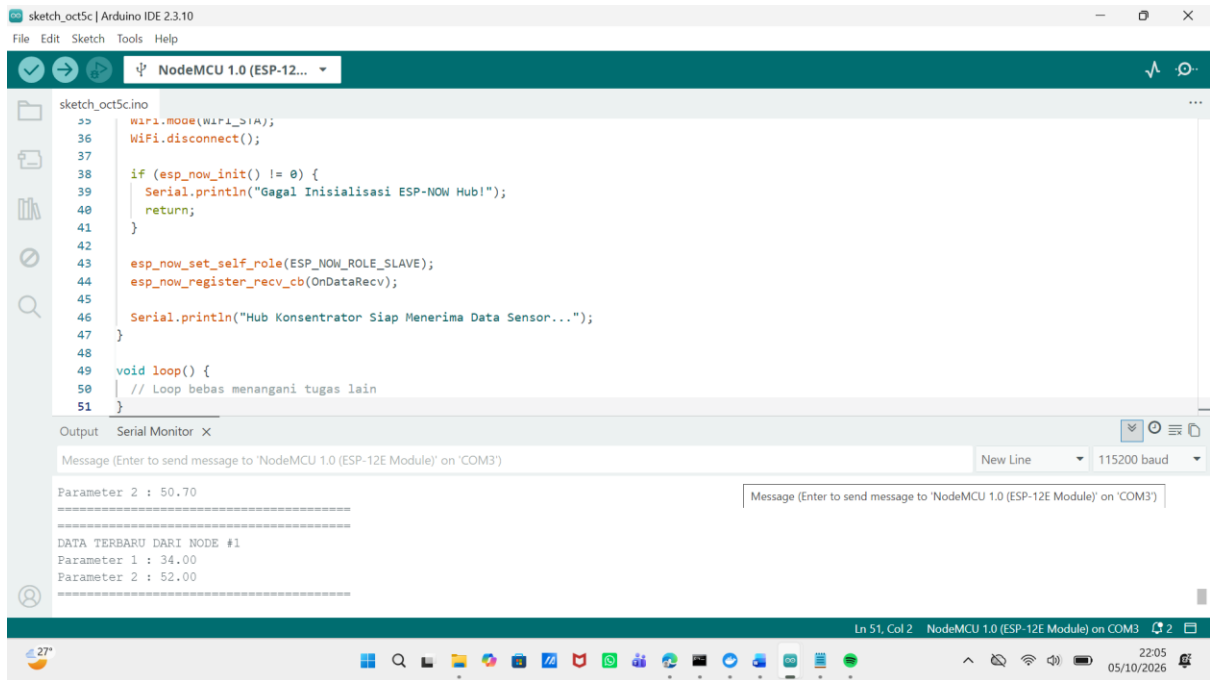
Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub

Ln 5, Col 62 NodeMCU 1.0 (ESP-12E Module) on COM3 2

Langkah Kerja Pemrograman Sisi Hub Konsentrator:



```
sketch_oct5c | Arduino IDE 2.3.10
File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...)

sketch_oct5c.ino
35  wifi.mode(WIFI_STA);
36  WiFi.disconnect();
37
38  if (esp_now_init() != 0) {
39    Serial.println("Gagal Inisialisasi ESP-NOW Hub!");
40    return;
41  }
42
43  esp_now_set_self_role(ESP_NOW_ROLE_SLAVE);
44  esp_now_register_recv_cb(OnDataRecv);
45
46  Serial.println("Hub Konsentrator Siap Menerima Data Sensor...");
47 }
48
49 void loop() {
50   // Loop bebas menangani tugas lain
51 }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

Parameter 2 : 50.70

=====

DATA TERBARU DARI NODE #1

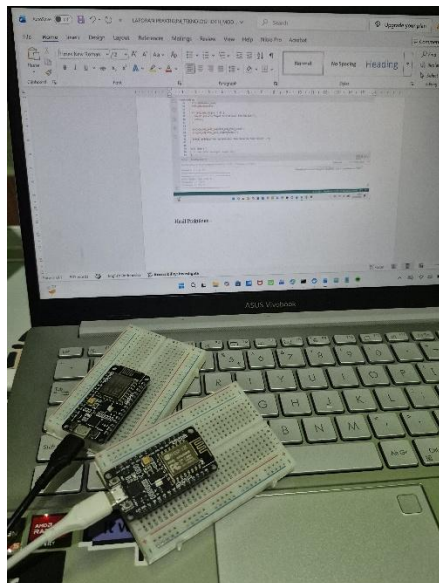
Parameter 1 : 34.00

Parameter 2 : 52.00

=====

Ln 51, Col 2 NodeMCU 1.0 (ESP-12E Module) on COM3 22:05 05/10/2026

Hasil Praktikum :

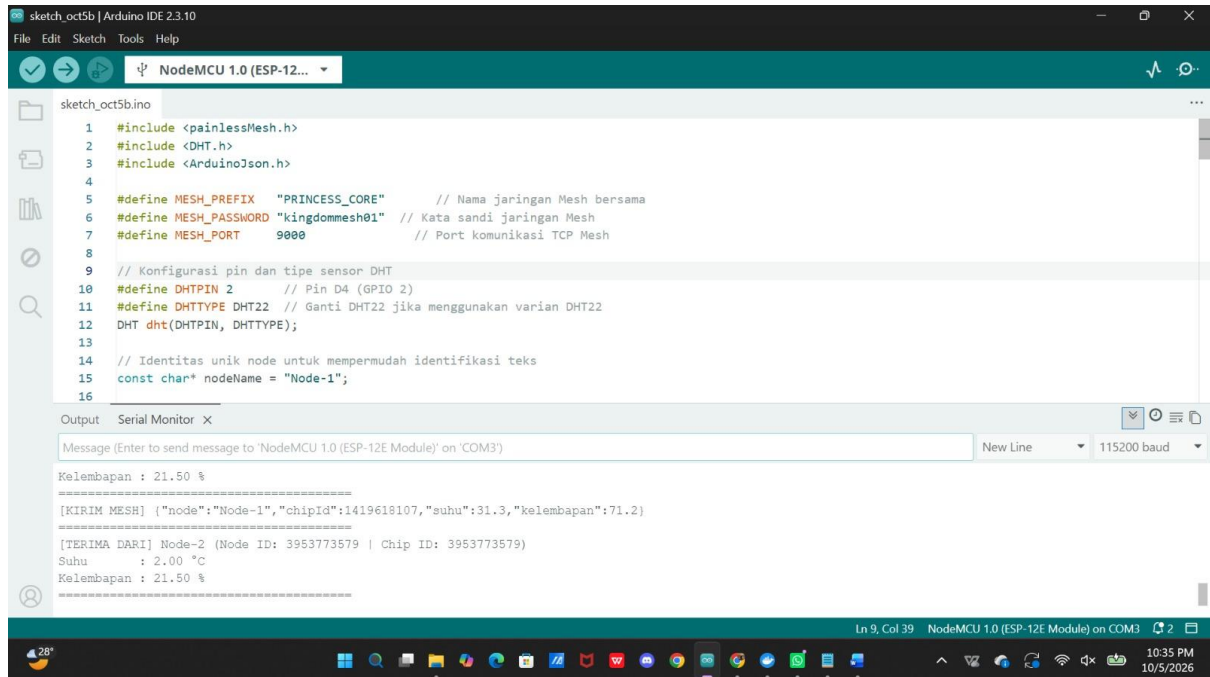


Secara keseluruhan, praktikum komunikasi nirkabel berbasis ESP-NOW membuktikan bahwa protokol ini mampu menghubungkan satu perangkat pengirim ke beberapa perangkat penerima secara nirkabel maupun mengumpulkan data telemetri dari berbagai node sensor menuju satu konsentrator pusat secara efisien tanpa memerlukan router Wi-Fi.

5.5.4 Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (painlessMesh)

Skenario: Seluruh board dalam kelompok dihubungkan ke sensor suhu & kelembapan riil (DHT11/DHT22) dan diintegrasikan ke dalam sebuah jaringan rantai otonom (*Mesh*). Setiap board secara berkala membaca sensor fisiknya, mengemas data telemetri ke dalam format JSON, dan menyiarkannya ke seluruh anggota jaringan tanpa perlu mengetahui alamat MAC masing-masing.

Pengirim Node-1



The screenshot shows the Arduino IDE interface with the following code in the sketch editor:

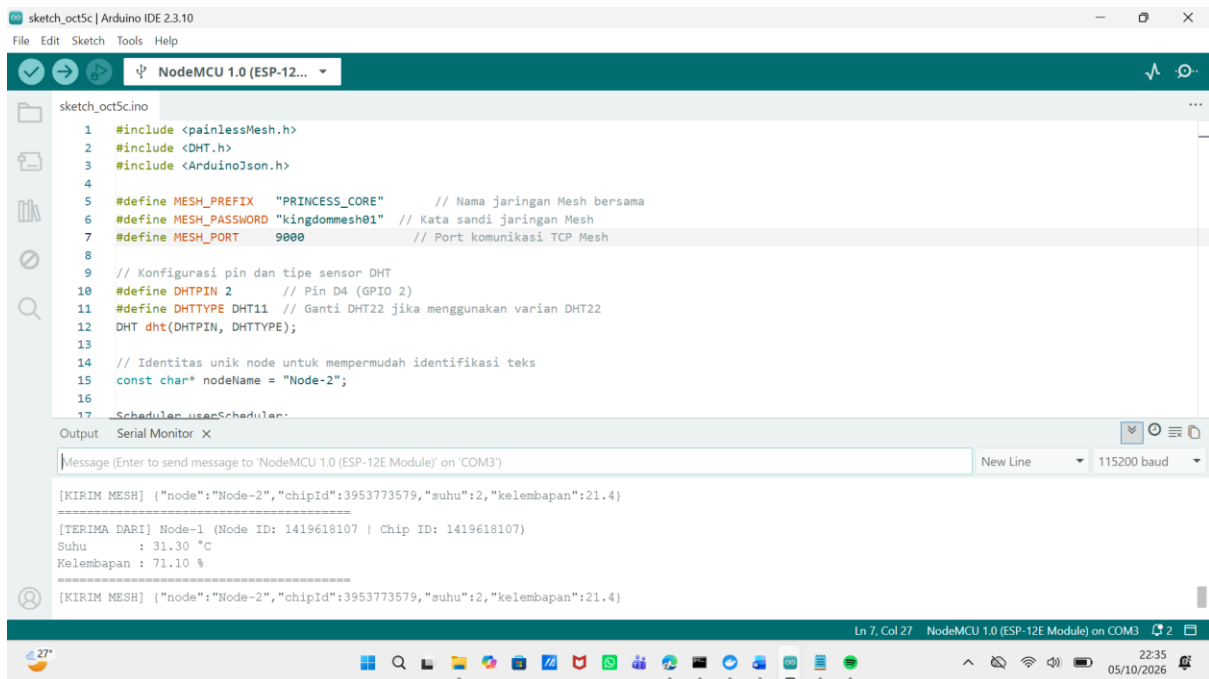
```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "PRINCESS_CORE" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "kingdommesh01" // Kata sandi jaringan Mesh
7 #define MESH_PORT 9000 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-1";
16
```

The Serial Monitor shows the following output:

```
Kelembapan : 21.50 %
=====
[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":31.3,"kelembapan":71.2}
=====
[TERIMA DARI] Node-2 (Node ID: 3953773579 | Chip ID: 3953773579)
Suhu      : 2.00 °C
Kelembapan : 21.50 %
```

The status bar at the bottom indicates the code is running on a NodeMCU 1.0 (ESP-12E Module) on COM3.

Pengirim Node-2



```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "PRINCESS_CORE" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "kingdommesh01" // Kata sandi jaringan Mesh
7 #define MESH_PORT 9000 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT11 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-2";
16
17 #include <Scheduler.h>
```

Output Serial Monitor X

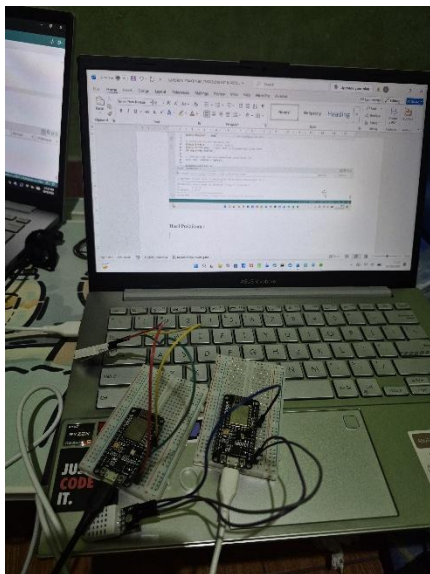
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

[KIRIM MESH] {"node": "Node-2", "chipId": 3953773579, "suhu": 2, "kelembapan": 21.4}

[TERIMA DARI] Node-1 (Node ID: 1419618107 | Chip ID: 1419618107)
Suhu : 31.30 °C
Kelembapan : 71.10 %

[KIRIM MESH] {"node": "Node-2", "chipId": 3953773579, "suhu": 2, "kelembapan": 21.4}

Hasil Praktikum :



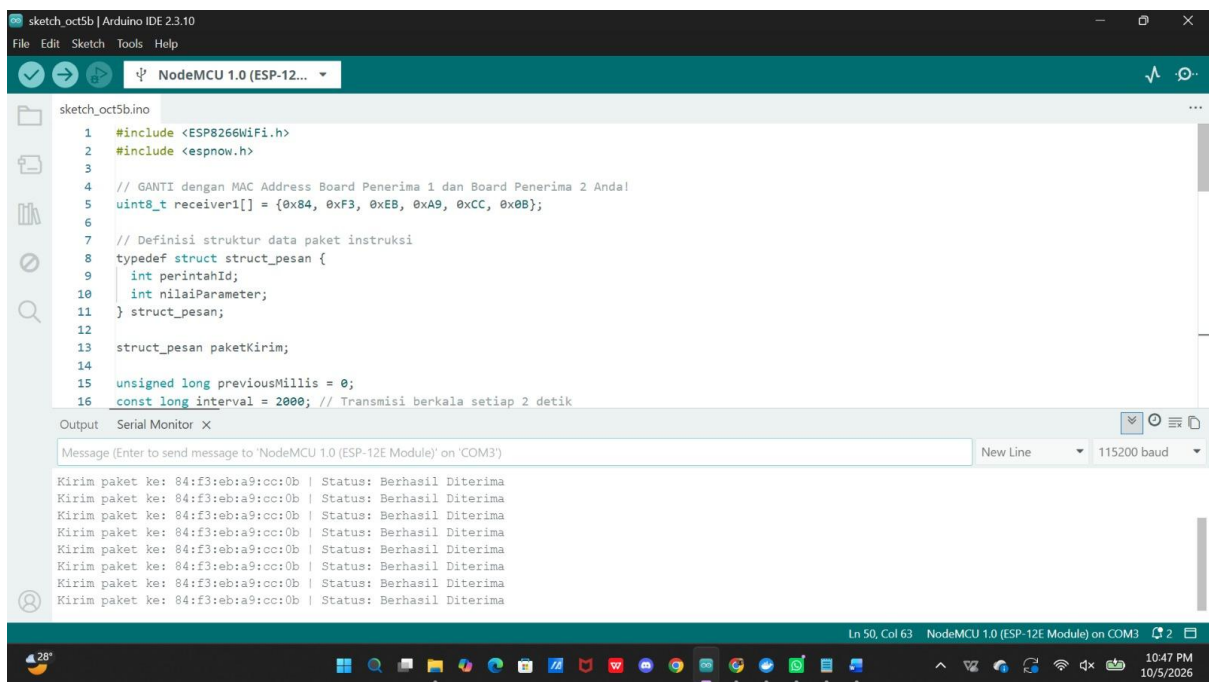
Penggunaan teknologi jaringan sensor ad-hoc multi-hop berbasis *painlessMesh* memungkinkan setiap node untuk saling terhubung secara otonom dan membentuk topologi rantai komunikasi nirkabel tanpa memerlukan perangkat pusat (*router* atau *gateway* Wi-Fi tradisional). Dengan mengintegrasikan sensor suhu dan kelembapan secara langsung ke pin GPIO board serta memanfaatkan format pertukaran data JSON, setiap node dapat melakukan pembacaan parameter fisik dan menyiarkannya secara transparan ke seluruh anggota jaringan secara mandiri dan dinamis.

5.6 Latihan

Kerjakan latihan berbasis eksperimen, observasi serial monitor, dan modifikasi kode berikut untuk melengkapi laporan praktikum Anda:

- Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2): Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status "Berhasil Diterima" (`sendStatus == 0`), lakukan eksperimen pemutusan simpul:
 - i. Cabut kabel USB salah satu board Penerima (misal NodeMCU 2) dari sumber daya.
 - ii. Amati perubahan log pada Serial Monitor NodeMCU Controller pengirim! Apakah status pengiriman untuk MAC address board yang dicabut berganti menjadi "Gagal (Tidak Terjangkau)"?
 - iii. Jelaskan bagaimana protokol ESP-NOW pada lapisan *Layer 2 MAC* dapat mendeteksi kegagalan transmisi ke perangkat fisik tersebut meskipun sistem tidak menggunakan koneksi TCP!

Node-1 Pengirim saat USB di colok :



```
sketch_oct5b.ino
1 #include <ESP8266WiFi.h>
2 #include <ESPNow.h>
3
4 // GANTI dengan MAC Address Board Penerima 1 dan Board Penerima 2 Anda!
5 uint8_t receiver1[] = {0x84, 0xF3, 0xEB, 0xA9, 0xCC, 0x0B};
6
7 // Definisi struktur data paket instruksi
8 typedef struct struct_pesan {
9     int perintahId;
10    int nilaiParameter;
11 } struct_pesan;
12
13 struct_pesan paketKirim;
14
15 unsigned long previousMillis = 0;
16 const long interval = 2000; // Transmisi berkala setiap 2 detik
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

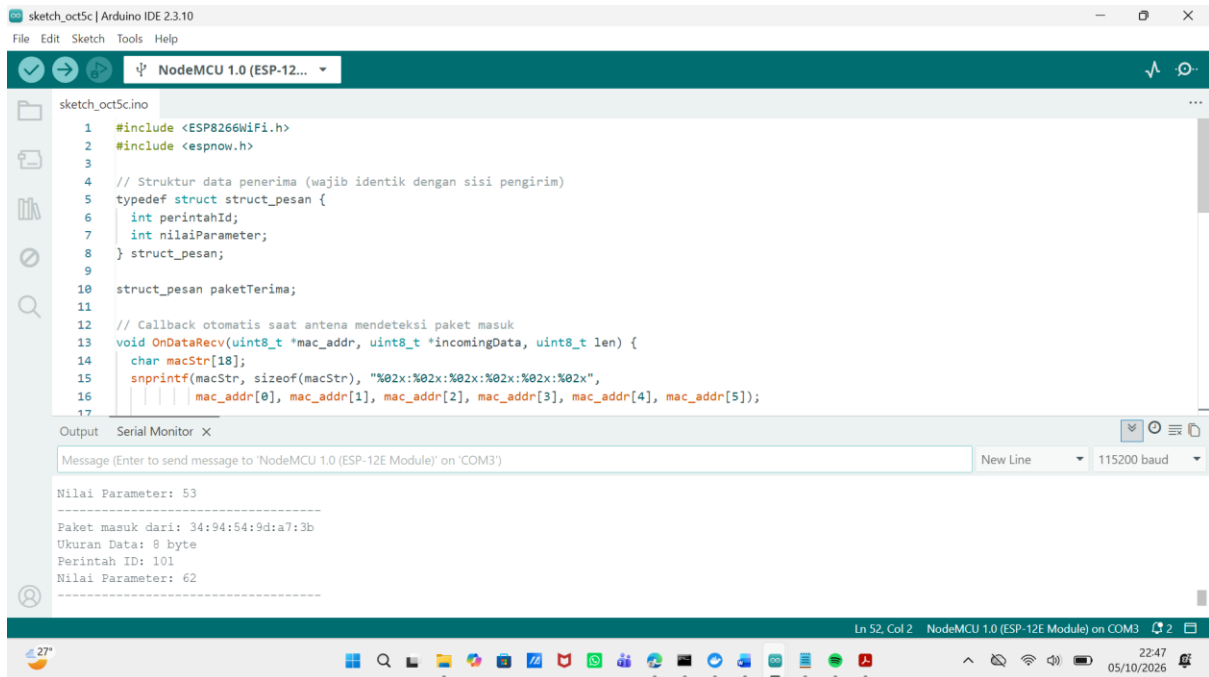
New Line 115200 baud

Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Berhasil Diterima

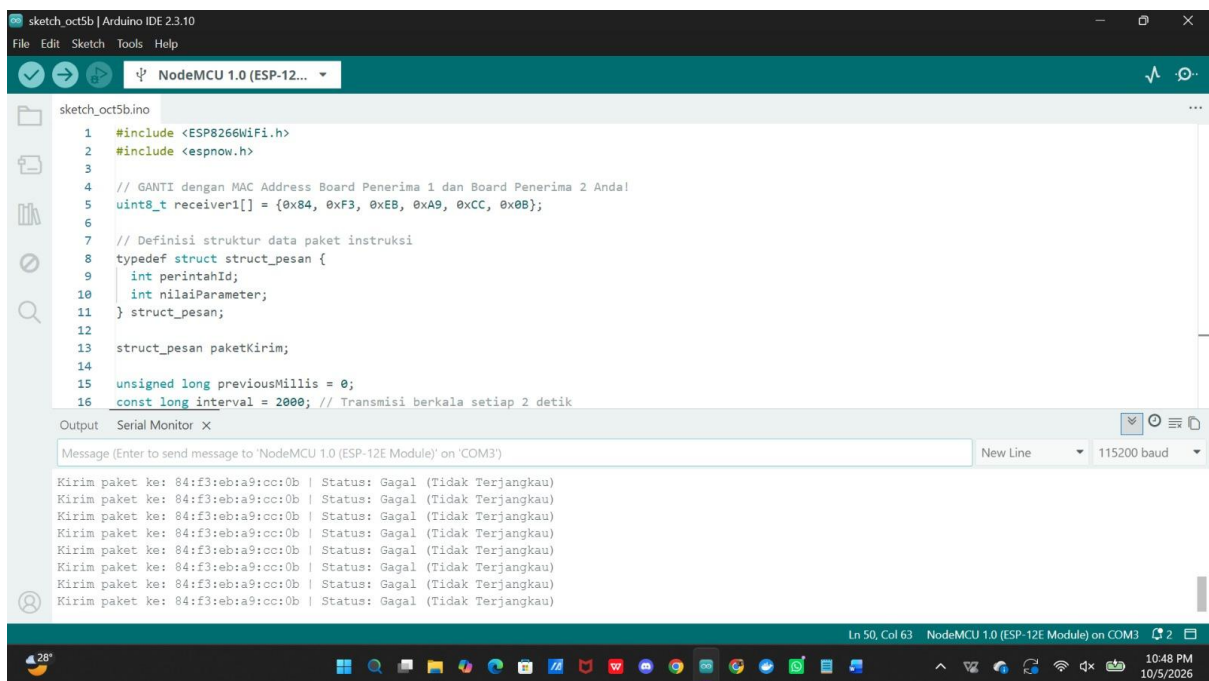
Ln 50, Col 63 NodeMCU 1.0 (ESP-12E Module) on COM3 2

10:47 PM 10/5/2026

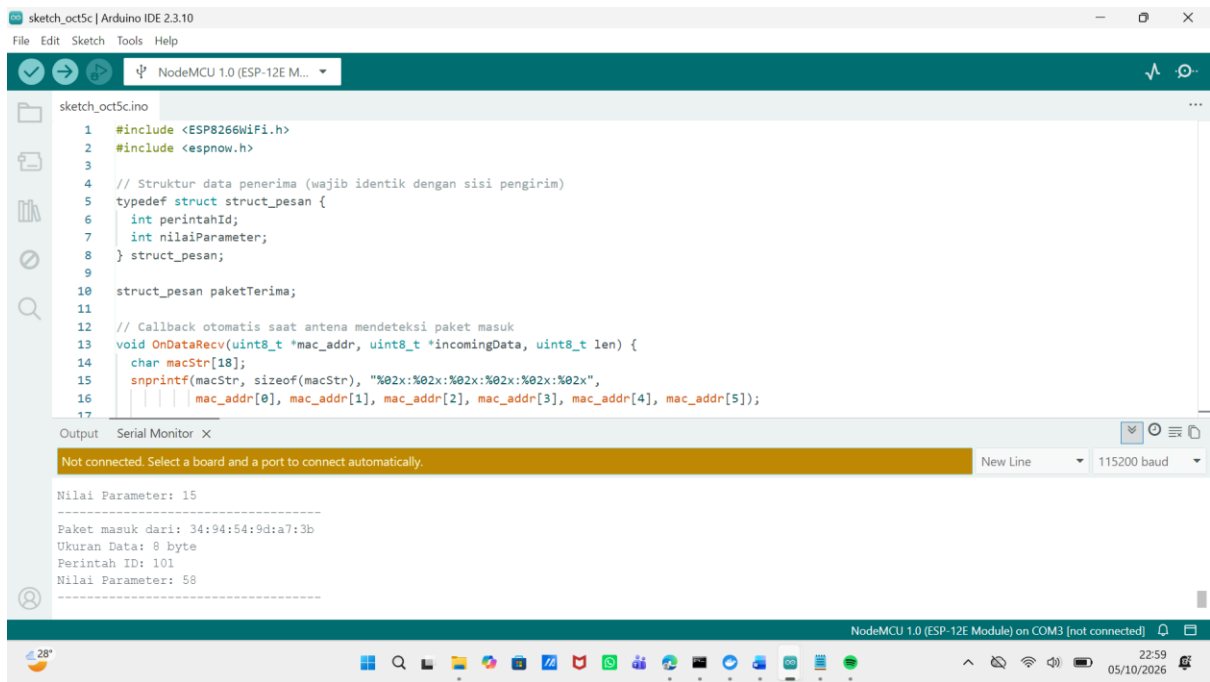
Node-2 Penerima paket :



Pengirim Node-1 :



Penerima Node-2 saat kabel USB di cabut :



```
sketch_oct5c| Arduino IDE 2.3.10
File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12E M...

sketch_oct5c.ino
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // Struktur data penerima (wajib identik dengan sisi pengirim)
5 typedef struct struct_pesan {
6     int perintahId;
7     int nilaiParameter;
8 } struct_pesan;
9
10 struct_pesan paketTerima;
11
12 // Callback otomatis saat antenna mendeteksi paket masuk
13 void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
14     char macStr[18];
15     snprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x:%02x",
16             mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);
17
18 Output Serial Monitor X
19 Not connected. Select a board and a port to connect automatically.
20 New Line 115200 baud
21
22 Nilai Parameter: 15
23 -----
24 Paket masuk dari: 34:94:54:9d:a7:3b
25 Ukuran Data: 8 byte
26 Perintah ID: 101
27 Nilai Parameter: 58
28 -----
29
30 NodeMCU 1.0 (ESP-12E Module) on COM3 [not connected]
31 22:59 05/10/2026
```

Berdasarkan eksperimen pemutusan simpul pada praktikum ESP-NOW One-to-Many, dapat disimpulkan bahwa saat salah satu board penerima dicabut dayanya, Serial Monitor pada board pengirim (Controller) akan mendeteksi kegagalan tersebut dan mengubah status pengiriman dari Berhasil Diterima menjadi Gagal atau Tidak Terjangkau. Meskipun ESP-NOW tidak menggunakan protokol TCP di Layer 4, kegagalan transmisi ini tetap dapat dideteksi karena ESP-NOW beroperasi langsung pada Layer 2 (Data Link/MAC Layer) dari standar Wi-Fi, di mana setiap pengiriman data wajib mendapatkan sinyal konfirmasi atau ACK (Acknowledgment) secara fisik dari perangkat penerima; jika perangkat penerima mati sehingga tidak ada ACK yang dikembalikan hingga batas waktu habis, sistem akan otomatis mencatat bahwa pengiriman ke MAC address tersebut mengalami kegagalan

- Latihan 2 (Eksperimen Dinamika Topologi & Self-Healing - Praktikum 4): Jalankan jaringan painlessMesh pada Praktikum 4 hingga seluruh board saling terhubung dan mencetak pesan --> Koneksi Baru Terdeteksi! Node ID:
 - i. Tekan tombol fisik RST atau cabut daya salah satu board Node. Amati Serial Monitor pada board lainnya yang masih menyala. Berapa detik waktu yang dibutuhkan oleh *callback* `changedConnectionCallback()` untuk mendeteksi bahwa salah satu simpul telah keluar dari rantai mesh?
 - ii. Sambungkan kembali daya board tersebut. Berapa lama waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh tanpa Anda mengonfigurasi ulang jaringan? Catat hasil observasi waktu tersebut!

Node-1 pengirim paket saat kabel USB di Node-2 di cabut

Tunggu sekitar 1 menit sampai ada output :

Topologi rantai telah di perbarui

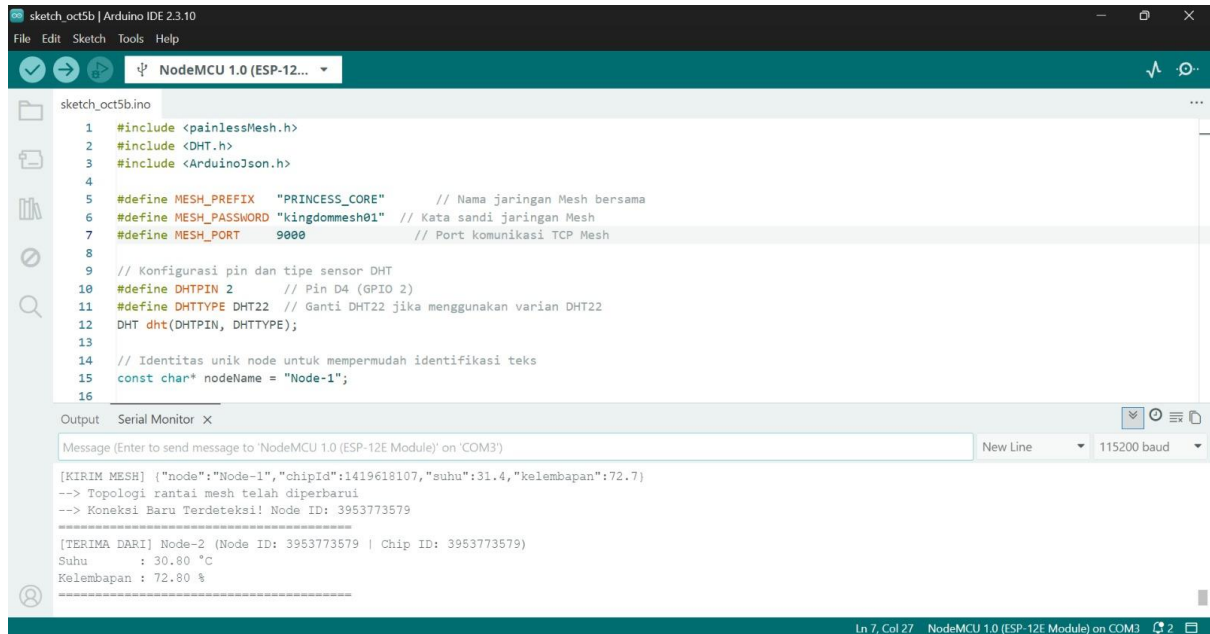
```

sketch_oct5b.ino
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "PRINCESS_CORE" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "kingdommesh01" // Kata sandi jaringan Mesh
7 #define MESH_PORT 9000 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-1";
16
Output Serial Monitor X
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')
New Line 115200 baud

[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":31.2,"kelembapan":72}
[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":31.2,"kelembapan":72}
[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":31.2,"kelembapan":72}
--> Topologi rantai mesh telah diperbarui
[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":31.2,"kelembapan":71.9}
[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":31.2,"kelembapan":72.1}
[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":31.2,"kelembapan":72}
[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":31.2,"kelembapan":72.5}
  
```

Kemudian saat Node-2 di colok USB kembali dia mengeluarkan output :

Koneksi Baru Terdeteksi! Node ID: 3953773579



The screenshot shows the Arduino IDE interface. The top menu bar includes File, Edit, Sketch, Tools, and Help. The toolbar contains icons for saving, running, and other functions. The sketch editor displays a file named 'sketch_oct5b.ino' with the following code:

```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "PRINCESS_CORE" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "kingdommesh01" // Kata sandi jaringan Mesh
7 #define MESH_PORT 9000 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-1";
16
```

The Serial Monitor window is open, showing the following output:

```
Message [Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3']
New Line 115200 baud

[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":31.4,"kelembapan":72.7}
--> Topologi rantai mesh telah diperbarui
--> Koneksi Baru Terdeteksi! Node ID: 3953773579

[TERIMA DARI] Node-2 (Node ID: 3953773579 | Chip ID: 3953773579)
Suhu : 30.80 °C
Kelembapan : 72.80 %
```

Waktu Deteksi Kehilangan Node (Disconnect): Ketika salah satu board Node dicabut dayanya atau ditekan tombol RST, callback `changedConnectionCallback()` pada board lainnya yang masih menyala membutuhkan waktu sekitar 1 menit untuk mendeteksi bahwa simpul tersebut telah keluar dari rantai mesh. Waktu ini dipengaruhi oleh interval pengiriman *heartbeat* (pesan pemeliharaan koneksi internal) antar node; jika node berhenti mengirimkan *heartbeat* hingga batas waktu *timeout* tertentu, jaringan secara otonom menyatakan node tersebut terputus.

Waktu Rekoneksi Otonom (Reconnect): Saat daya board dihidupkan kembali, node tersebut membutuhkan waktu sekitar 1 menit untuk kembali terdaftar secara otonom (*self-healing*) ke dalam topologi mesh tanpa perlu melakukan konfigurasi ulang jaringan (seperti pengaturan SSID, password, ataupun *re-flashing* program). Node secara otomatis melakukan *scanning* dan menyambung kembali ke jalur rute terdekat yang tersedia di dalam mesh.

- **Latihan 3 (Eksperimen Modifikasi Filter Telemetri & Ambang Batas JSON - Praktikum 4):** Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetri dari semua pengirim tanpa penyaringan.
 - i. Modifikasilah fungsi `receivedCallback` pada salah satu board agar bertindak sebagai Pengawas Kondisi Suhu Kritis: program HANYA mencetak kartu telemetri penuh ke Serial Monitor jika suhu yang dilaporkan oleh node tetangga bernilai di atas 32.0 °C (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim).
 - ii. Jika suhu bernilai normal (≤ 32.0 °C), program cukup mencetak status singkat satu baris: `[NORMAL] Telemetri dari Node <chipId> aman.`
 - iii. Tuliskan blok fungsi `receivedCallback(uint32_t from, String &msg)` hasil modifikasi Anda!

Node-1 Pengirim:

```

1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "PRINCESS_CORE" // Nama jaringan Mesh
6 #define MESH_PASSWORD "kingdommesh01" // Kata sandi Mesh
7 #define MESH_PORT 9000 // Port komunikasi TCP
8
9 // Konfigurasi pin dan tipe sensor DHT22
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Menggunakan sensor DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node (Ubah "Node-1" pada board pertama, dan "Node-2" pada board kedua)
15 const char* nodeName = "Node-1";
16
17 void receivedCallback(uint32_t from, String &msg) {
18   // Parse JSON message
19   DynamicJsonDocument doc(1024);
20   deserializeJson(doc, msg);
21   JsonObject root = doc.as<JsonObject>();
22   const char* node = root["node"];
23   const float suhu = root["suhu"];
24   const float kelembapan = root["kelembapan"];
25
26   // Check if temperature is above 32.0°C
27   if (suhu > 32.0) {
28     // Print full JSON telemetry
29     Serial.println(msg);
30   } else {
31     // Print short status message
32     Serial.println("[NORMAL] Telemetri dari Node " + node + " aman.");
33   }
34 }
35
36 void setup() {
37   Serial.begin(115200);
38   mesh.begin();
39   mesh.config(MESH_PREFIX, MESH_PASSWORD, MESH_PORT);
40   mesh.addNode(nodeName);
41   pinMode(DHTPIN, INPUT);
42 }
43
44 void loop() {
45   mesh.update();
46   if (mesh.isNodeConnected(nodeName)) {
47     String msg = mesh.receiveFrom(nodeName);
48     receivedCallback(mesh.getNodeId(nodeName), msg);
49   }
50   dht.read();
51   Serial.println("Suhu: " + String(dht.temperature()) + "°C, Kelembapan: " + String(dht.humidity()) + "%");
52 }

```

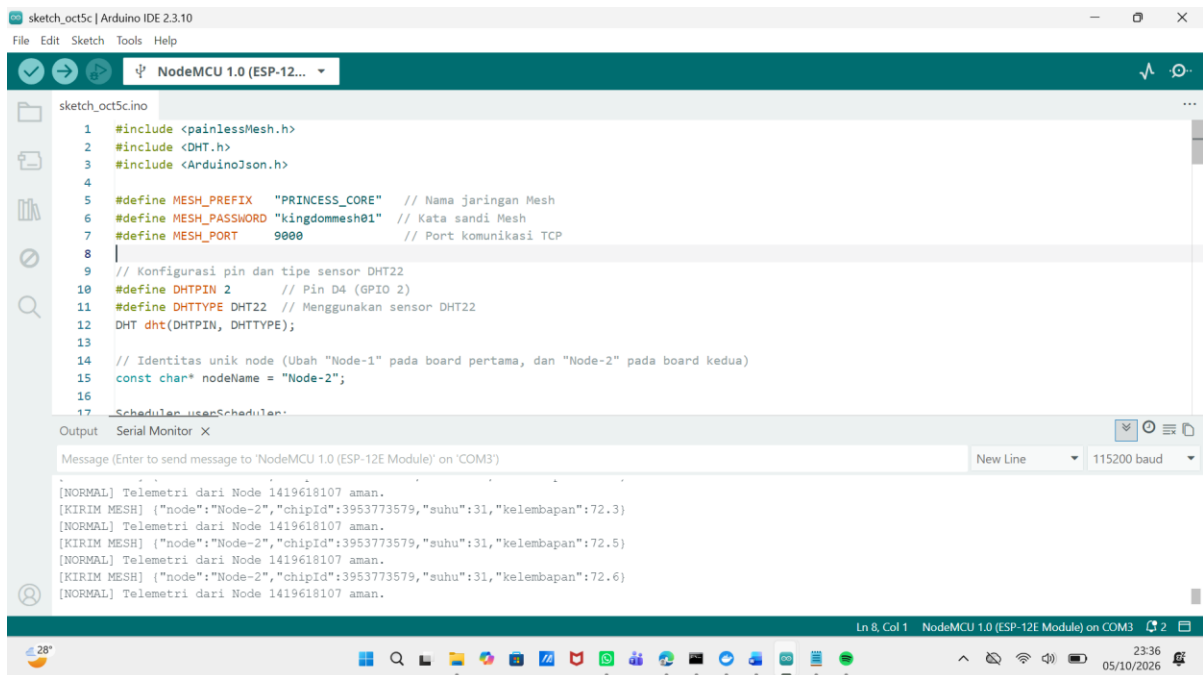
Output Serial Monitor:

```

[NORMAL] Telemetri dari Node 3953773579 aman.
[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":31,"kelembapan":72.1}
[NORMAL] Telemetri dari Node 3953773579 aman.
[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":31.1,"kelembapan":73.2}
[NORMAL] Telemetri dari Node 3953773579 aman.
[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":31.1,"kelembapan":72.6}
[NORMAL] Telemetri dari Node 3953773579 aman.
[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":31.1,"kelembapan":72.3}

```

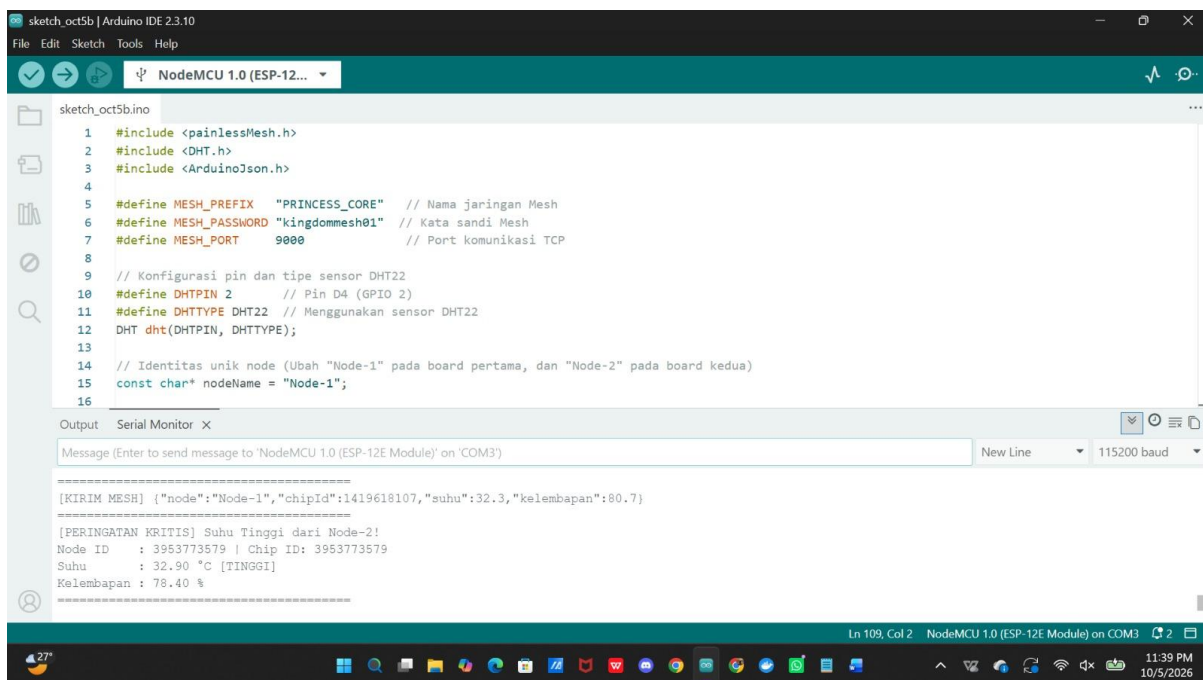
Node-2 Penerima :



The screenshot shows the Arduino IDE interface with the sketch 'sketch_oct5c.ino' open. The code defines a Node-2 receiver for a mesh network. The serial monitor displays the following output:

```
[NORMAL] Telemetri dari Node 1419618107 aman.  
[KIRIM MESH] {"node":"Node-2","chipId":3953773579,"suhu":31,"kelembapan":72.3}  
[NORMAL] Telemetri dari Node 1419618107 aman.  
[KIRIM MESH] {"node":"Node-2","chipId":3953773579,"suhu":31,"kelembapan":72.5}  
[NORMAL] Telemetri dari Node 1419618107 aman.  
[KIRIM MESH] {"node":"Node-2","chipId":3953773579,"suhu":31,"kelembapan":72.6}  
[NORMAL] Telemetri dari Node 1419618107 aman.
```

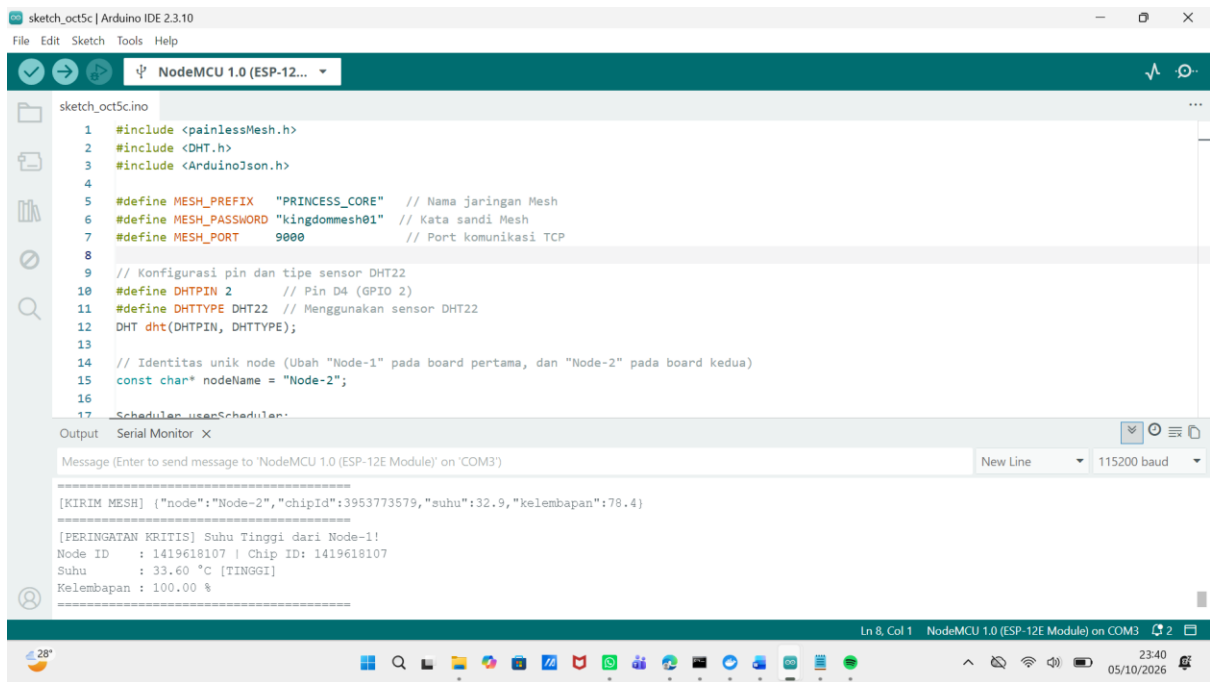
Saat sensor DHT si tiup di Node-2 output Node-1 berubah menjadi Node-2 :



The screenshot shows the Arduino IDE interface with the sketch 'sketch_oct5b.ino' open. The code defines a Node-1 receiver for a mesh network. The serial monitor displays the following output:

```
[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":32.3,"kelembapan":80.7}  
[PERINGATAN KRITIS] Suhu Tinggi dari Node-2!  
Node ID : 3953773579 | Chip ID: 3953773579  
Suhu : 32.90 °C [TINGGI]  
Kelembapan : 78.40 %
```

Begitu pula Node-2 saat DHT di tiup dia otomatis ke Node-1:



```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "PRINCESS_CORE" // Nama jaringan Mesh
6 #define MESH_PASSWORD "kingdommesh01" // Kata sandi Mesh
7 #define MESH_PORT 9000 // Port komunikasi TCP
8
9 // Konfigurasi pin dan tipe sensor DHT22
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Menggunakan sensor DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node (Ubah "Node-1" pada board pertama, dan "Node-2" pada board kedua)
15 const char* nodeName = "Node-2";
16
17 #include <Scheduler.h>
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

[KIRIM MESH] {"node":"Node-2","chipId":3953773579,"suhu":32.9,"kelembapan":78.4}

[PERINGATAN KRITIS] Suhu Tinggi dari Node-1!

Node ID : 1419618107 | Chip ID: 1419618107

Suhu : 33.60 °C [TINGGI]

Kelembapan : 100.00 %

Berdasarkan eksperimen modifikasi filter telemetry dan ambang batas JSON pada jaringan painlessMesh, dapat disimpulkan bahwa fungsi `receivedCallback` dapat diprogram untuk menyaring data masuk menggunakan pustaka `ArduinoJson`. Melalui penerapan kondisi logika ambang batas, sistem berhasil membedakan status telemetry secara real-time—mencetak kartu telemetry penuh secara rinci hanya ketika suhu dari node pengirim melewati batas kritis ($>32.0\text{ }^{\circ}\text{C}$), dan menampilkan status singkat satu baris ketika kondisi lingkungan berada dalam rentang normal ($\leq 32.0\text{ }^{\circ}\text{C}$). Hal ini efektif mengurangi kepadatan log pada Serial Monitor sekaligus meningkatkan efisiensi pemantauan berbasis IoT.

Tuliskan blok fungsi `receivedCallback(uint32_t from, String &msg)` hasil modifikasi Anda!

```
void receivedCallback(uint32_t from, String &msg) {
    // Parsing muatan data JSON yang diterima dari jaringan mesh
    StaticJsonDocument<200> doc;
    DeserializationError error = deserializeJson(doc, msg);

    if (!error) {
        const char* sender = doc["node"];
        uint32_t chipId = doc["chipId"];
        float suhu = doc["suhu"];
```

```

float kelembapan = doc["kelembapan"];

// Filter ambang batas suhu kritis (> 32.0 °C)
if (suhu > 32.0) {
    Serial.println("=====");
    Serial.printf("[PERINGATAN KRITIS] Suhu Tinggi dari %s!\n", sender);
    Serial.printf("Node ID   : %u | Chip ID: %u\n", from, chipId);
    Serial.printf("Suhu       : %.2f °C [TINGGI]\n", suhu);
    Serial.printf("Kelembapan : %.2f %%\n", kelembapan);
    Serial.println("=====");
} else {
    // Tampilan status singkat jika suhu normal (<= 32.0 °C)
    Serial.printf("[NORMAL] Telemetri dari Node %u aman.\n", chipId);
}
} else {
    Serial.printf("[ERROR JSON DARI %u]: %s\n", from, msg.c_str());
}
}

```